EKS setup — monitoring, logging, addons & ArgoCD onboarding

Source: Abhishek Ashok Kumar Upadhyay (internal doc). Single file — core addons, Kyverno policies, logging (Fluent Bit/Fluentd), monitoring (kube-prometheus-stack), AlertManager routing, OIDC, ArgoCD cluster onboarding, sanity script, IAM notes.

---

§1 — What this covers (high-level)

A production-ready EKS cluster needs more than just the control plane. This document walks through every layer you bolt on after the cluster exists:

- Core addons — networking (VPC CNI), DNS (CoreDNS), proxying (kube-proxy), storage (EBS CSI).
- Policy engine — Kyverno cluster policies (namespace isolation, resource limits, image pull).
- Logging — Fluent Bit / Fluentd daemonsets shipping logs to your backend.
- Monitoring — kube-prometheus-stack (Prometheus, Grafana, node-exporter, AlertManager).
- Alerting — AlertManager with VictorOps receivers routed by namespace.
- OIDC — IAM identity provider so pods can assume AWS roles (required for Karpenter, ALB controller, etc.).
- ArgoCD onboarding — registering a new cluster into your central ArgoCD (Mumbai cluster).
- Sanity check script — automated verification of all the above.

[Diagram -- see markdown source for mermaid]

---

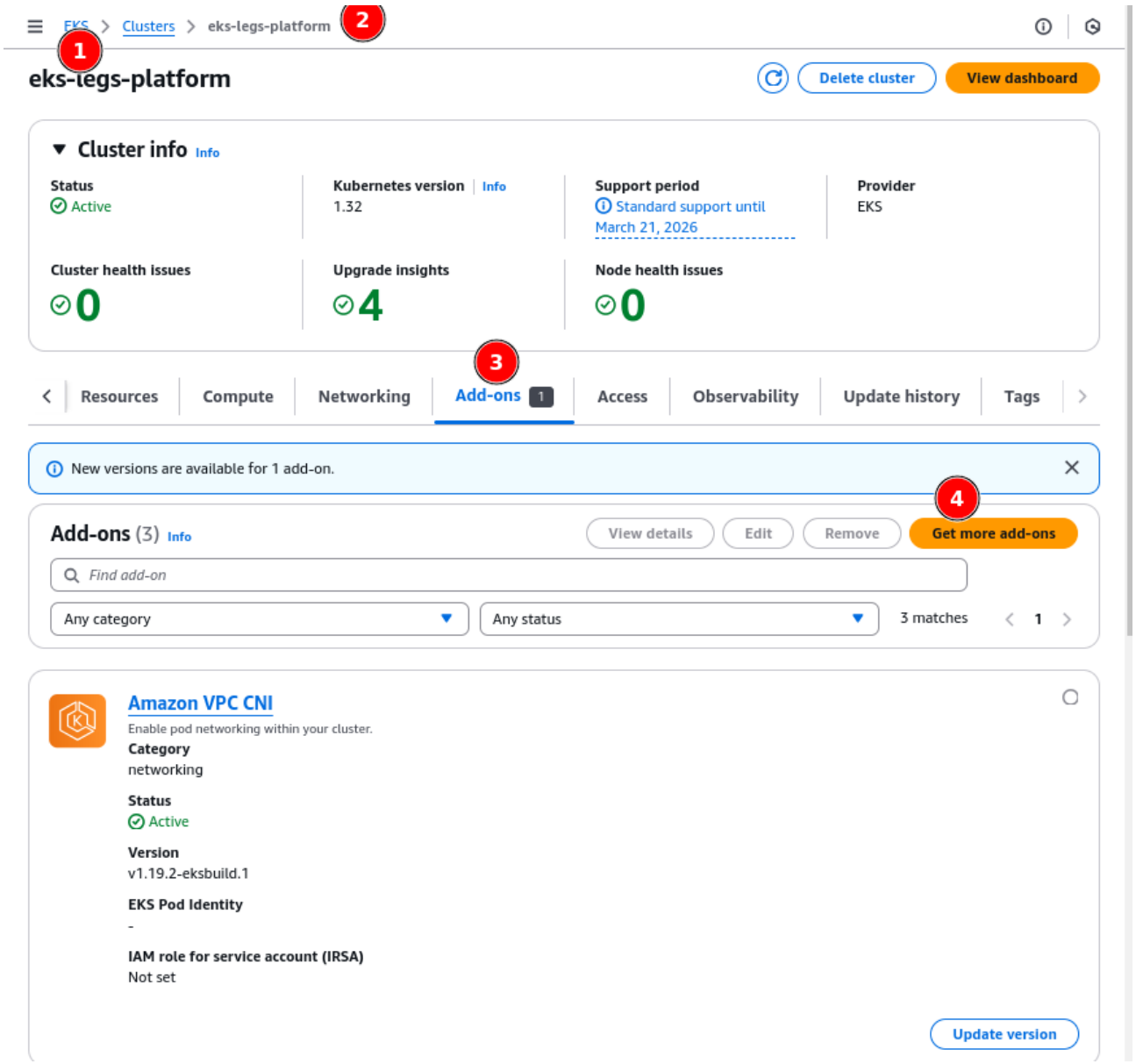
§2 — Core addons

These are installed via the AWS EKS console > Addons tab (or Terraform/CDK). Behind the scenes EKS uses Helm.

Addon	What it does	Config notes
Amazon VPC CNI	Pod networking — assigns ENI IPs to pods.	Add {"enableNetworkPolicy": "true"} in advanced config; choose override on conflict.
CoreDNS	Cluster DNS (service discovery).	Default config is fine initially.
kube-proxy	iptables / IPVS rules for Service → Pod traffic.	Default config.
EBS CSI Driver	Allows PVCs to provision EBS volumes.	Needs OIDC + IAM role (§8).

Steps:

- EKS Console → Add-ons → select the four addons above.



EKS Addons setup — Add-ons tab showing VPC CNI, CoreDNS, kube-proxy, EBS CSI

2. Use the latest compatible version for each.
3. Wait until each addon shows Active status before proceeding.
4. Verify with Helm:

```
helm3 ls -A
```

All four should appear as releases.

---

§3 — Kyverno (policy engine)

Kyverno enforces cluster policies as Kubernetes-native resources (no external webhook config needed beyond the controller).

Install

Navigate to your kubernetes-charts/kyverno folder and follow the README.md:

```
cd kubernetes-charts/kyverno
# install Kyverno controller per README
kubectl get cpol -A   # verify CRDs exist
```

Apply policies

```
cd kyverno/policies
for f in $(ls *.yaml); do
  echo "Applying $f"
  kubectl apply -f "$f"
done
```

Required policies

Policy	What it enforces
disallow-default-namespace	Blocks workloads in default namespace.
enforce-network-policy	Auto-creates a NetworkPolicy in every new namespace.
require-requests-limits	Every container must declare CPU/memory requests and limits.
set-image-pull-policy	Forces Always pull policy (prevents stale cached images).

Verify with NetworkPolicy test

```
# Label existing namespaces so the policy allows inter-ns traffic
for ns in $(kubectl get ns --no-headers | awk '{print $1}'); do
  kubectl label ns "$ns" application-type=system --overwrite
done

# Create test namespaces
kubectl create ns foobar
kubectl create ns spamegg

# Check auto-created NetworkPolicy
kubectl get networkpolicies -n foobar
kubectl get networkpolicies -n spamegg
# expect: allow-namespace-communication-and-internet
```

Connectivity test

- Create a pod in foobar, another in spamegg.
- In pod A: nc -lvp 1234
- In pod B: nc <pod-A-IP> 1234
- Without application-type=system label → connection hangs (blocked).
- With the label → connection succeeds.

---

§4 — Logging (Fluent Bit + Fluentd)

Logging uses Fluent Bit (lightweight DaemonSet collector) and Fluentd (aggregator/router) shipping to your log backend (OpenSearch, S3, etc.).

Install

Apply the YAML from your repository's eks-legs-platform folder:

```
kubectl apply -f fluentd-logging/
```

Checklist

- [ ] Fluent Bit DaemonSet running on all nodes: kubectl get ds -n logging
- [ ] Fluentd pods running: kubectl get pods -n logging
- [ ] ConfigMap present — verify it's not missed during apply: kubectl get cm -n logging
- [ ] Logs flowing to backend — check OpenSearch / your destination

Reference: internal Fluentd Logging docs in the repo.

---

§5 — EBS CSI Driver + gp3-retain StorageClass

The Helm chart hardcodes gp3-retain as the StorageClass. Retain reclaim policy means if a PVC is deleted, the underlying EBS volume is not deleted — critical for Prometheus metrics data that you can't afford to lose.

Create the StorageClass

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: gp3-retain
parameters:
  fsType: ext4
  type: gp3
provisioner: kubernetes.io/aws-ebs
reclaimPolicy: Retain
volumeBindingMode: WaitForFirstConsumer
```

```
kubectl apply -f gp3-retain-sc.yaml
kubectl get sc gp3-retain
```

Why Retain: Prometheus, Loki, and other observability stores keep their data on PVCs. If someone accidentally deletes a PVC or a Helm uninstall removes it, Retain keeps the EBS volume so data can be recovered by creating a new PV pointing to the same volume ID.

---

§6 — Monitoring stack (kube-prometheus-stack)

Uses the kube-prometheus-stack Helm chart which bundles:

Component	What it does
Prometheus	Metrics collection and storage (TSDB).
Grafana	Dashboards and visualization.
node-exporter	DaemonSet — host-level metrics (CPU, RAM, disk, network) from every node.
kube-state-metrics	Kubernetes object metrics (pod status, deployment replicas, etc.).
AlertManager	Alert routing and notification (§7).
Prometheus Operator	Manages Prometheus/AlertManager CRDs (ServiceMonitor, PrometheusRule, etc.).

Install

```
cd kubernetes-charts/kube-prometheus-stack

# Dry-run: review generated YAML
helm3 template --namespace monitoring kube-prometheus-stack \
  ./kube-prometheus-stack -f kube-prometheus-stack/values.yaml --debug

# Install / upgrade
helm3 upgrade --namespace monitoring --install kube-prometheus-stack \
```

```
./kube-prometheus-stack -f kube-prometheus-stack/values.yaml --debug
```

Post-install verification

```
kubectl get pods -n monitoring          # all Running / Completed
kubectl get svc -n monitoring           # Prometheus, Grafana, AlertManager services
kubectl get prometheusrules -n monitoring
kubectl get servicemonitors -n monitoring
```

metrics-server (required for HPA + CPU metrics)

```
kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml
kubectl get deployment metrics-server -n kube-system
kubectl top nodes   # should return CPU/memory after ~60s
```

---

§7 — AlertManager configuration

AlertManager is installed by the Helm chart but ships without routing config — it's not functional until you create the routing rules.

Architecture

[Diagram -- see markdown source for mermaid]

Step 1: Create receiver secrets

Each receiver gets its own Secret containing the VictorOps (or other) API key. Keeping them separate prevents one bad edit from breaking all receivers.

```
apiVersion: v1
kind: Secret
metadata:
  name: infra-receiver-secret-am
  namespace: monitoring
type: Opaque
data:
  api-key: <base64-encoded-api-key>
```

Create similar secrets for: default-receiver-secret-am, data-receiver-secret-am, billing-receiver-secret-am.

```
kubectl apply -f alertmanager-secrets/
```

Step 2: Create AlertmanagerConfig

```
apiVersion: monitoring.coreos.com/v1alpha1
kind: AlertmanagerConfig
metadata:
  labels:
    release: kube-prometheus-stack
  name: custom-alertmanager-config
  namespace: monitoring
spec:
  receivers:
    - name: default
      victoropsConfigs:
        - apiKey:
            key: api-key
            name: default-receiver-secret-am
```

```
        routingKey: route-infra-engg-sev0
        sendResolved: true
        entityDisplayName: >-
          [EKS <cluster-name>] {{ .CommonLabels.namespace }}
          - {{ .CommonLabels.alertname }}
          - {{ .CommonLabels.severity | toUpper }}
        messageType: '{{ .CommonLabels.severity | toUpper }}'
        stateMessage: |
          ALERT: {{ .CommonLabels.alertname }}
          MESSAGE: {{ .CommonAnnotations.message }}
          NAMESPACE: {{ .CommonLabels.namespace }}

  - name: infra-receiver
    victoropsConfigs:
      - apiKey:
          key: api-key
          name: infra-receiver-secret-am
        routingKey: route-infra-engg-sev0
        sendResolved: true
        entityDisplayName: >-
          [EKS <cluster-name> {INFRA}] {{ .CommonLabels.namespace }}
          - {{ .CommonLabels.alertname }}
          - {{ .CommonLabels.severity | toUpper }}
        messageType: '{{ .CommonLabels.severity | toUpper }}'
        stateMessage: |
          ALERT: {{ .CommonLabels.alertname }}
          MESSAGE: {{ .CommonAnnotations.message }}
          NAMESPACE: {{ .CommonLabels.namespace }}

  - name: data-receiver
    victoropsConfigs:
      - apiKey:
          key: api-key
          name: data-receiver-secret-am
        routingKey: route-infra-engg-sev0
        sendResolved: true
        entityDisplayName: >-
          [EKS <cluster-name> {DATA}] {{ .CommonLabels.namespace }}
          - {{ .CommonLabels.alertname }}
          - {{ .CommonLabels.severity | toUpper }}
        messageType: '{{ .CommonLabels.severity | toUpper }}'
        stateMessage: |
          ALERT: {{ .CommonLabels.alertname }}
          MESSAGE: {{ .CommonAnnotations.message }}
          NAMESPACE: {{ .CommonLabels.namespace }}

  - name: billing-receiver
    victoropsConfigs:
      - apiKey:
          key: api-key
          name: billing-receiver-secret-am
        routingKey: route-infra-engg-sev0
        sendResolved: true
        entityDisplayName: >-
          [EKS <cluster-name> {BILLING}] {{ .CommonLabels.namespace }}
          - {{ .CommonLabels.alertname }}
          - {{ .CommonLabels.severity | toUpper }}
        messageType: '{{ .CommonLabels.severity | toUpper }}'
        stateMessage: |
          ALERT: {{ .CommonLabels.alertname }}
          MESSAGE: {{ .CommonAnnotations.message }}
          NAMESPACE: {{ .CommonLabels.namespace }}

route:
  receiver: infra-receiver
  groupBy: [alertname, service, namespace, metateam, hostname, consumergroup, topic]
  groupWait: 10s
  groupInterval: 1m
  repeatInterval: 10m
  routes:
    # --- Infra namespaces ---
```

```
      - matchers: [{name: namespace, value: logstash, matchType: "="}]
        receiver: infra-receiver
      - matchers: [{name: namespace, value: frodo, matchType: "="}]
        receiver: infra-receiver
      - matchers: [{name: namespace, value: infra, matchType: "="}]
        receiver: infra-receiver
      - matchers: [{name: namespace, value: monitoring, matchType: "="}]
        receiver: infra-receiver
      - matchers: [{name: namespace, value: metrics, matchType: "="}]
        receiver: infra-receiver
      - matchers: [{name: namespace, value: kube-system, matchType: "="}]
        receiver: infra-receiver
      - matchers: [{name: namespace, value: kube-public, matchType: "="}]
        receiver: infra-receiver
      - matchers: [{name: namespace, value: default, matchType: "="}]
        receiver: infra-receiver
      - matchers: [{name: namespace, value: cert-manager, matchType: "="}]
        receiver: infra-receiver
      - matchers: [{name: namespace, value: argocd, matchType: "="}]
        receiver: infra-receiver
      - matchers: [{name: namespace, value: foobar, matchType: "="}]
        receiver: infra-receiver
      - matchers: [{name: namespace, value: istio-system, matchType: "="}]
        receiver: infra-receiver
      - matchers: [{name: namespace, value: starix, matchType: "="}]
        receiver: infra-receiver
      # --- Data namespaces ---
      - matchers: [{name: namespace, value: call-es-worker-callmetric, matchType: "="}]
        receiver: data-receiver
      - matchers: [{name: namespace, value: call-es-worker-callinbox, matchType: "="}]
        receiver: data-receiver
      - matchers: [{name: namespace, value: jellibabix, matchType: "="}]
        receiver: data-receiver
      - matchers: [{name: namespace, value: appengine-mysqlstreamer, matchType: "="}]
        receiver: data-receiver
      - matchers: [{name: namespace, value: twlx-mysqlstreamer, matchType: "="}]
        receiver: data-receiver
      - matchers: [{name: namespace, value: reportix, matchType: "="}]
        receiver: data-receiver
      # --- Billing namespaces ---
      - matchers: [{name: namespace, value: billixasync, matchType: "="}]
        receiver: billing-receiver
      - matchers: [{name: namespace, value: approvalsystem, matchType: "="}]
        receiver: billing-receiver
      - matchers: [{name: namespace, value: paymentsystem, matchType: "="}]
        receiver: billing-receiver
      - matchers: [{name: namespace, value: creditcontrol, matchType: "="}]
        receiver: billing-receiver
      - matchers: [{name: namespace, value: invoicesystem, matchType: "="}]
        receiver: billing-receiver
      - matchers: [{name: namespace, value: priceupdater, matchType: "="}]
        receiver: billing-receiver
      - matchers: [{name: namespace, value: daakiyatrix, matchType: "="}]
        receiver: billing-receiver
      - matchers: [{name: namespace, value: billixcb, matchType: "="}]
        receiver: billing-receiver
      - matchers: [{name: namespace, value: fulliautomatix, matchType: "="}]
        receiver: billing-receiver
      - matchers: [{name: namespace, value: billingaddendum, matchType: "="}]
        receiver: billing-receiver
```

```
kubectl apply -f custom-alertmanager-config.yaml
```

Step 3: Verify

```
kubectl get alertmanagerconfig -n monitoring
kubectl get alertmanager -n monitoring

# Check operator logs for rejected configs
kubectl logs -f deployment/kube-prometheus-stack-operator -n monitoring | grep 'rejected'
```

```
# empty output = no config errors
```

Routing logic summary

Receiver	Namespaces
infra-receive	logstash, frodo, infra, monitoring, metrics, kube-system, kube-public, default, cert-manager, argocd, foobar, istio-system, starix
r data-receive	call-es-worker-callmetric, call-es-worker-callinbox, jellibabix, appengine-mysqlstreamer, twlx-mysqlstreamer, reportix
billing-receiv er	billixasync, approvalsystem, paymentsystem, creditcontrol, invoicesystem, priceupdater, daakiyatrix, billixcb, fulliautomatix, billingaddendum

Unmatched namespaces fall through to the top-level infra-receiver.

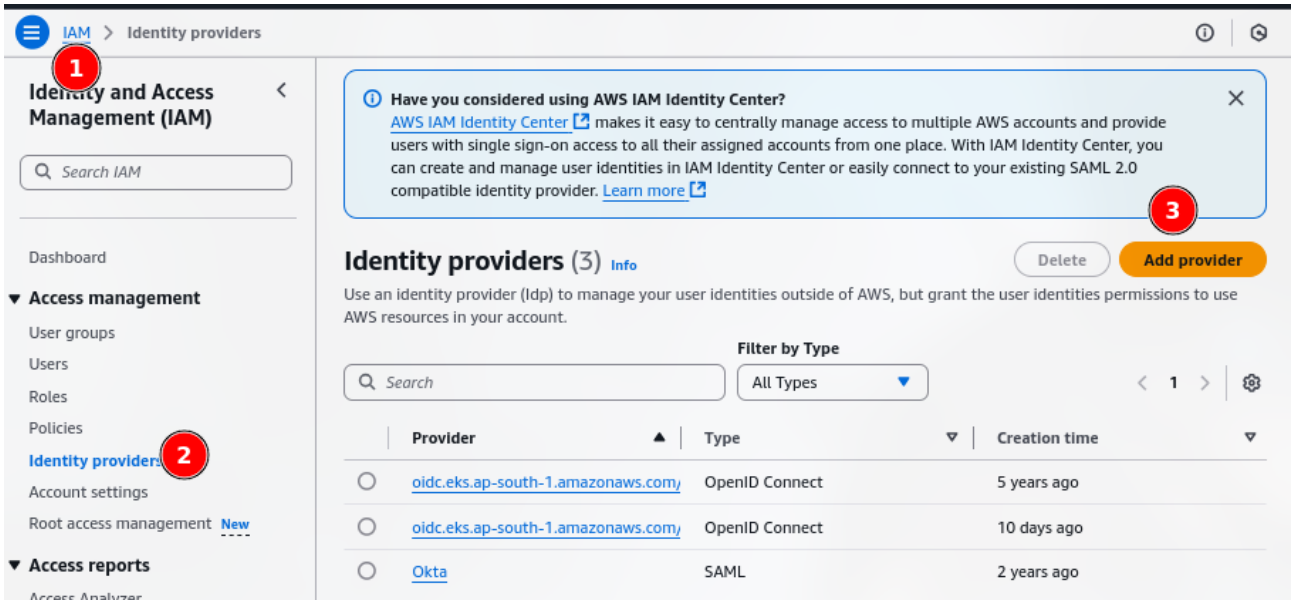
---

§8 — OIDC setup (IAM roles for pods)

OIDC lets EKS pods assume IAM roles — required for any component that talks to AWS APIs (Karpenter, ALB Ingress Controller, EBS CSI, S3 access, etc.).

Steps

- IAM Console → Identity providers → Add provider.



IAM Identity Providers — OIDC providers for EKS

2. Provider type: OpenID Connect.
3. Provider URL: your cluster's OIDC issuer — format:

```
oidc.eks.<region>.amazonaws.com/id/<CLUSTER_OIDC_ID>
```

Example: oidc.eks.ap-south-1.amazonaws.com/id/D1F46832E89C057D1A91AF404C108D53

- Audience: sts.amazonaws.com
- Save.

IAM role trust policy (for a pod to assume a role)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
```

```
      "Effect": "Allow",
      "Principal": {
        "Federated": "arn:aws:iam::<ACCOUNT_ID>:oidc-provider/oidc.eks.<REGION>.amazonaws.com/id/<OIDC_ID>"
      },
      "Action": "sts:AssumeRoleWithWebIdentity",
      "Condition": {
        "StringEquals": {
          "oidc.eks.<REGION>.amazonaws.com/id/<OIDC_ID>:sub": "system:serviceaccount:<NAMESPACE>:<SA_NAME>"
        }
      }
    }
  ]
}
```

Naming conventions

Where	Name pattern	Example
Kubernetes ServiceAccount	<application>-pod-role	wanda-pod-role (no cluster suffix)
AWS IAM role	<application>-<cluster-identifier>	wanda-legsplatform (cluster suffix present — shared platform, multiple clusters need unique names)

Key notes

- Trust relationships must be reviewed carefully — understand what each condition in the trust policy means before changing it.
- IAM role needs kms:* permission if the application decrypts secrets (KMS encryption).

---

§9 — ArgoCD cluster onboarding

This registers a new target cluster into your central ArgoCD running on the Mumbai cluster.

Terminology

Term	Meaning
Mumbai cluster	EKS cluster where ArgoCD runs (control plane).
Target cluster	The new EKS cluster being onboarded.
argocd-manager	ServiceAccount created on the target cluster for ArgoCD access.

Prerequisites

- Kubernetes >= 1.24
- ArgoCD installed and healthy on Mumbai cluster
- kubectl access to both clusters
- argocd CLI installed and authenticated
- Admin-level access on the target cluster

9.1 Sanity check (Mumbai cluster)

```
kubectl -n argocd get pods              # all Running / Completed, no CrashLoopBackOff
argocd cluster list                     # CLI can reach ArgoCD server
```

9.2 Prepare the target cluster

All commands below run against the target cluster context.

Create ServiceAccount:

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: argocd-manager
  namespace: kube-system
```

```
kubectl apply -f argocd-manager-sa.yaml
kubectl -n kube-system get sa argocd-manager
```

Create ClusterRoleBinding:

roleRef is immutable — if a binding with the same name exists, delete it first or use a different name.

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: argocd-manager-cluster-admin
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: cluster-admin
subjects:
  - kind: ServiceAccount
    name: argocd-manager
    namespace: kube-system
```

```
kubectl apply -f argocd-manager-crb.yaml
kubectl get clusterrolebinding argocd-manager-cluster-admin
```

Create ServiceAccount token (K8s >= 1.24):

```
apiVersion: v1
kind: Secret
metadata:
  name: argocd-manager-token
  namespace: kube-system
  annotations:
    kubernetes.io/service-account.name: argocd-manager
type: kubernetes.io/service-account-token
```

```
kubectl apply -f argocd-manager-token.yaml
kubectl -n kube-system get secret argocd-manager-token
```

9.3 Register cluster in ArgoCD (Mumbai cluster)

```
# Confirm context points to target cluster
kubectl config current-context

# Add cluster
argocd cluster add <target-cluster-context> \
  --service-account argocd-manager \
  --system-namespace kube-system
```

9.4 Verify

```
argocd cluster list
# STATUS: Successful, CONNECTION STATE: Successful

kubectl -n argocd get secrets | grep cluster
# look for secret with label: argocd.argoproj.io/secret-type=cluster
```

9.5 Troubleshooting

Error	Cause	Fix
cannot change roleRef	Tried to modify existing ClusterRoleBinding.	Delete and recreate, or use a new binding name.
Cluster status Unknown / Failed	Missing/invalid SA token, insufficient RBAC, wrong kubeconfig context.	Verify SA exists, token secret exists, CRB is correct; re-run argocd cluster add.

---

§10 — Sanity check script

Run this after completing all sections to verify the cluster is production-ready.

```
#!/bin/bash
set -euo pipefail

RED='\033[0;31m'
GREEN='\033[0;32m'
NC='\033[0m'
X="${RED}${NC}"
CHECK="${GREEN}${NC}"

CRITICAL_NAMESPACES=(
  default
  autoscaler
  kube-system
  os-metrics
  kube-node-lease
  foobar
  monitoring
  kube-public
)

# --- kube-system pods ---
echo " Checking required pods in kube-system..."
declare -A LABELS_TO_CHECK=(
  ["k8s-app"]="aws-node kube-dns kube-proxy"
  ["app"]="ebs-csi-controller ebs-csi-node efs-csi-controller efs-csi-node"
)
for label_key in "${!LABELS_TO_CHECK[@]}"; do
  for label_val in ${LABELS_TO_CHECK[$label_key]}; do
    echo -n "   $label_key=$label_val... "
    if kubectl get pods -n kube-system -l "$label_key=$label_val" --no-headers 2>/dev/null | grep -q .; then
      echo -e "$CHECK Found"
    else
      echo -e "$X Not found"
    fi
  done
done
echo

# --- Kyverno ClusterPolicies ---
echo " Checking ClusterPolicies..."
REQUIRED_CPOL_NAMES=(
  "disallow-default-namespace"
  "enforce-network-policy"
  "require-requests-limits"
  "set-image-pull-policy"
)
for cpol in "${REQUIRED_CPOL_NAMES[@]}"; do
  echo -n "   $cpol... "
  if kubectl get cpol "$cpol" -o jsonpath='{.status.ready}' 2>/dev/null | grep -q "True"; then
    echo -e "$CHECK Ready"
  else
    echo -e "$X Missing or not Ready"
  fi
```

```
done
echo

# --- NetworkPolicies + namespace labels ---
echo " Checking NetworkPolicy and labels in critical namespaces..."
EXPECTED_NETPOL="allow-namespace-communication-and-internet"
for ns in "${CRITICAL_NAMESPACES[@]}"; do
  echo -n "   [$ns] NetworkPolicy... "
  if kubectl get netpol "$EXPECTED_NETPOL" -n "$ns" --no-headers &>/dev/null; then
    echo -e "$CHECK Found"
  else
    echo -e "$X Missing"
  fi
  echo -n "     [$ns] Label application-type=system... "
  label_val=$(kubectl get ns "$ns" -o jsonpath='{.metadata.labels.application-type}' 2>/dev/null || echo "")
  if [[ "$label_val" == "system" ]]; then
    echo -e "$CHECK Present"
  else
    echo -e "$X Missing"
  fi
done
```

What it checks

Check	What it verifies
kube-system pods	aws-node (VPC CNI), kube-dns (CoreDNS), kube-proxy, EBS CSI controller/node, EFS CSI controller/node
ClusterPolicies	All four Kyverno policies are present and Ready
NetworkPolicies	allow-namespace-communication-and-internet exists in every critical namespace
Namespace labels	application-type=system label on every critical namespace

---

§11 — Additional addons (reference list)

These are mentioned as part of a complete setup but installed separately per their own docs:

Addon	Purpose
Loki	Log aggregation (pairs with Grafana for log querying).
istiod (Istio)	Service mesh / ingress controller.
os-metrics	OpenSearch metrics collection.
Cluster Autoscaler	Scales node groups based on pending pods.
k8s-spot-termination-handler	Graceful drain on spot instance interruption.
Karpenter	Node provisioning (alternative to Cluster Autoscaler). Needs OIDC (§8).
AWS ALB Ingress Controller	Provisions ALBs for Ingress resources. Needs OIDC (§8).
cert-manager	TLS certificate management (Let's Encrypt, internal CA).

---

§12 — Key notes and gotchas

ServiceAccount naming on EKS: <application>-pod-role (e.g. wanda-pod-role) — no cluster suffix. On the AWS IAM side, include a cluster identifier (e.g. wanda-legsplatform) because multiple EKS clusters share the same AWS account.

IAM trust relationships: Review every field — understand what StringEquals conditions on the OIDC subject mean. A wrong condition silently denies AssumeRoleWithWebIdentity.

KMS permissions: If the application reads encrypted data (Secrets Manager, S3 SSE-KMS, etc.), the IAM role needs kms:Decrypt (or kms:* if you're not restricting).

AlertManager CRD changes: The AlertmanagerConfig CRD is relatively new. If upgrading kube-prometheus-stack versions, run kubectl get crd alertmanagerconfigs.monitoring.coreos.com -o yaml to check for schema changes before updating your config.

EKS addons vs Helm: EKS-managed addons auto-upgrade within your chosen channel. If you need pinned versions or custom values, manage them as self-managed Helm releases instead.

gp3-retain StorageClass: Always use for observability data (Prometheus, Loki). Delete reclaim policy = data gone on PVC deletion.

Single file. Order: core addons → policies → logging → storage → monitoring → alerting → OIDC → ArgoCD → sanity check → addons list → key notes.